

Backpropagation and Optimizers

Due: Friday, November 7th at 11:59 pm

Deliverables. Submit a PDF of your write-up to Gradescope *HW3 Written*

Note: we **highly discourage** very long answers, your free response answers should be 1-3 sentences.

Honor Code

Write and sign the following statement:

“I certify that all solutions in this document are entirely my own and that I have not looked at anyone else’s solution. I have given credit to all external sources I consulted.”

AI assistance disclosure: OpenAI Codex assisted with explanations, solution drafting, implementation, and verification. The honor-code statement above is part of the assignment template and has not been signed.

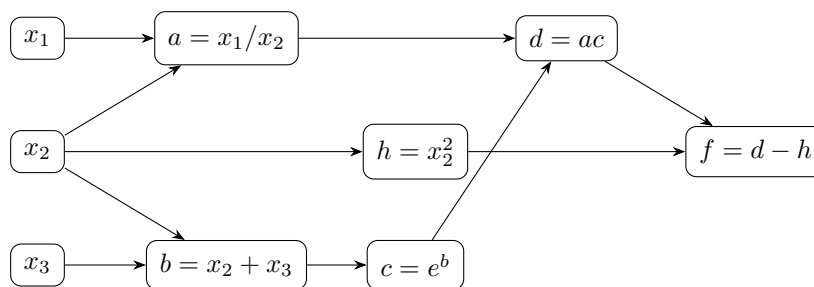
Backpropagation Practice

Let's practice calculating the gradient using adjoints. We're given the following function and want to find the gradient of the function with respect $\mathbf{x} = [x_1 \ x_2 \ x_3]$:

$$f(\mathbf{x}) = \frac{x_1}{x_2} e^{x_2+x_3} - x_2^2$$

Q1. Constructing the Computation Graph Draw the computation graph (check out lecture 12 for reference). We recommend handwriting it instead of using L^AT_EX.

Solution. Define $a = x_1/x_2$, $b = x_2 + x_3$, $c = e^b$, $d = ac$, and $h = x_2^2$, so that $f = d - h$.



Q2. Computing Adjoints Now, compute the adjoint for each node in your computation graph with respect to $\mathbf{x} = [x_1 \ x_2 \ x_3] = [1 \ 2 \ 3]$. What is the gradient with respect to $\mathbf{x}$? We recommend that you use your computation graph and write it out instead of typing.

Solution. Let $\bar{v} = \partial f / \partial v$. The forward values are

$$a = \frac{1}{2}, \quad b = 5, \quad c = e^5, \quad d = \frac{1}{2}e^5, \quad h = 4, \quad f = \frac{1}{2}e^5 - 4.$$

Starting with $\bar{f} = 1$, reverse propagation gives

$$\bar{d} = 1, \quad \bar{h} = -1, \quad \bar{a} = \bar{d}c = e^5, \quad \bar{c} = \bar{d}a = \frac{1}{2}, \quad \bar{b} = \bar{c}e^b = \frac{1}{2}e^5.$$

The input adjoints are obtained by summing over all outgoing paths:

$$\begin{aligned} \bar{x}_1 &= \bar{a} \frac{1}{x_2} = \frac{e^5}{2}, \\ \bar{x}_2 &= \bar{a} \left(-\frac{x_1}{x_2^2} \right) + \bar{b} + \bar{h}(2x_2) = -\frac{e^5}{4} + \frac{e^5}{2} - 4 = \frac{e^5}{4} - 4, \\ \bar{x}_3 &= \bar{b} = \frac{e^5}{2}. \end{aligned}$$

Thus,

$$\nabla_{\mathbf{x}} f(1, 2, 3) = \begin{pmatrix} e^5/2 \\ e^5/4 - 4 \\ e^5/2 \end{pmatrix} \approx \begin{pmatrix} 74.2066 \\ 33.1033 \\ 74.2066 \end{pmatrix}.$$

Optimizers Overview

Adaptive gradient methods are among the most widely used optimization algorithms in deep learning. In this assignment, we focus on two main algorithms: **Adam** and **AdamW** (briefly AdaGrad, RMSProp). Because the primary sources are dense, please start with the short blog posts below, then (optionally) consult the original papers.

Recommended reading (short):

- Understanding Deep Learning Optimizers: Momentum, AdaGrad, RMSProp, Adam — quick refresher on the progression from SGD to Adam.
- Adam vs. AdamW (A. Bennaceur) — concise comparison highlighting why decoupled weight decay matters.
- **Supplement (Section 3):** CS182 Discussion 2 Solutions (Spring 2021) — see Section 3 for a compact optimizer summary.

Optional primary sources:

- Adam: A Method for Stochastic Optimization (Kingma & Ba, 2015)
- Decoupled Weight Decay Regularization (Loshchilov & Hutter, 2017)

Before answering the questions, also briefly review the original Adam algorithm (Algorithm 1 from Kingma & Ba, 2014) and AdamW, reproduced below for reference.

Algorithm 1 Adam, our proposed algorithm for stochastic optimization (Kingma & Ba, 2014)

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

```

1:  $m_0 \leftarrow 0$                                 ▷ Initialize 1st moment vector
2:  $v_0 \leftarrow 0$                                 ▷ Initialize 2nd moment vector
3:  $t \leftarrow 0$                                 ▷ Initialize timestep
4: while  $\theta_t$  not converged do
5:    $t \leftarrow t + 1$ 
6:    $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$           ▷ Get gradients
7:    $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$   ▷ EMA of gradients
8:    $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$   ▷ EMA of squared gradients
9:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$               ▷ Bias-corrected 1st moment
10:   $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$               ▷ Bias-corrected 2nd moment
11:   $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$   ▷ Update
12: end while
13: return  $\theta_t$ 

```

Algorithm 2 Adam with L2 regularization and Adam with decoupled weight decay (AdamW)**Require:** $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\lambda \in \mathbb{R}$

- 1: **initialize** $t \leftarrow 0$, parameter vector $\theta_{t=0} \in \mathbb{R}^n$, first moment $m_{t=0} \leftarrow 0$, second moment $v_{t=0} \leftarrow 0$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
- 2: **repeat**
- 3: $t \leftarrow t + 1$
- 4: $\nabla f_t(\theta_{t-1}) \leftarrow \text{SELECTBATCH}(\theta_{t-1})$
- 5: $g_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$ ▷ (L2-regularized Adam) add penalty inside gradient
- 6: $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
- 7: $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
- 8: $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$
- 9: $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$
- 10: $\eta_t \leftarrow \text{SETSCHEDULEMULTIPLIER}(t)$
- 11: $\theta_t \leftarrow \theta_{t-1} - \eta_t \left(\alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_{t-1} \right)$ ▷ (AdamW) decoupled decay
- 12: **until** stopping criterion is met
- 13: **return** optimized parameters θ_t

Q3. Reviewing Previous Optimizers. We have learned how variants of gradient descent improve training speed and stability. For each method — **Momentum**, **AdaGrad**, and **RMSPprop** — write the update equations, state the key idea (one concise sentence), and describe the problem it solves (one concise sentence). We have provided SGD for you.

SGD

Update: $\theta_t \leftarrow \theta_{t-1} - \alpha g_t$, where $g_t = \nabla_{\theta} f_t(\theta_{t-1})$.

Key idea: Use a random minibatch each step for computational efficiency.

Problem it solves: Full-batch gradient descent is slow and redundant; minibatching makes updates cheaper.

Solution. All operations below are elementwise, $g_t = \nabla f_t(\theta_{t-1})$, and all accumulators start at zero.

Momentum:

$$u_t = \beta u_{t-1} + g_t, \quad \theta_t = \theta_{t-1} - \alpha u_t.$$

Key idea: Accumulate an exponentially decaying history of gradients. *Problem it solves:* It reduces oscillation and accelerates progress along directions with consistent gradients.

AdaGrad:

$$s_t = s_{t-1} + g_t^2, \quad \theta_t = \theta_{t-1} - \alpha \frac{g_t}{\sqrt{s_t} + \epsilon}.$$

Key idea: Scale each coordinate's update by its accumulated squared gradients. *Problem it solves:* It adapts to unequal gradient scales and gives infrequently updated coordinates relatively larger effective learning rates.

RMSProp:

$$s_t = \beta s_{t-1} + (1 - \beta)g_t^2, \quad \theta_t = \theta_{t-1} - \alpha \frac{g_t}{\sqrt{s_t} + \epsilon}.$$

Key idea: Use an exponential moving average of squared gradients to adapt each coordinate's step size. *Problem it solves:* It avoids AdaGrad's continually growing accumulator and the resulting continual decay of effective learning rates.

Q4. Handling Large Gradients. What happens when gradients are very large in plain gradient descent, and how does Adam's update rule address this issue?

Solution. Large gradients in plain gradient descent can cause oversized updates, overshooting, oscillation, or divergence. Adam smooths gradients with momentum and divides the update by $\sqrt{\hat{v}_t} + \epsilon$, reducing the effective learning rate in coordinates with large recent squared gradients; this mitigates oversized steps but does not guarantee stability.

Q5. Introducing AdamW. How does AdamW decouple weight decay from the gradient update? Why does this help generalization?

Solution. AdamW computes Adam's moments using only the loss gradient and applies weight decay separately:

$$\theta_t = (1 - \alpha\lambda)\theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

The decay is not accumulated in the moments or rescaled by the adaptive denominator, so it provides more predictable regularization that can discourage large weights and reduce overfitting. Here λ follows the learning-rate-scaled convention in Q8.

Q6. AdamW Weight Decoupling. How does decoupling weight decay affect hyperparameter sensitivity?

Solution. Decoupling prevents the strength of weight decay from depending on Adam’s gradient history and coordinatewise scaling, making the decay coefficient easier to tune separately from the adaptive update. This reduces hyperparameter interactions, but does not eliminate them: under the convention in Q8, the per-step shrinkage still depends on $\alpha\lambda$.

Q7. Tracing Adam’s Origins (Line Mapping). As seen in lecture, Adam builds on ideas from several earlier optimizers. Now let’s explore exactly how these ideas are used in Adam. Below are the key lines (7–11) from the Adam algorithm.

For each line, label which previous algorithm inspired it (or indicate if it was newly introduced) using one of the following categories:

- Momentum
- Adaptive Scaling (RMSProp/AdaGrad)
- Newly Introduced by Adam
- Combination (merges two or more earlier ideas into a single update rule)

Briefly explain your reasoning for each label.

Algorithm 3 Adam: Core Update Lines (Kingma & Ba, 2014)

1: $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$	▷ Line 7
2: $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$	▷ Line 8
3: $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$	▷ Line 9
4: $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$	▷ Line 10
5: $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$	▷ Line 11

Solution.

- Line 7: **Momentum**; it accumulates an exponential moving average of gradients.
- Line 8: **Adaptive Scaling (RMSProp/AdaGrad)**; the moving average of squared gradients supplies coordinatewise scale estimates.
- Line 9: **Newly Introduced by Adam** (among the compared methods); it corrects the first moment’s initial bias toward zero.
- Line 10: **Newly Introduced by Adam** (among the compared methods); it corrects the second moment’s initial bias toward zero.
- Line 11: **Combination**; it combines the momentum numerator with adaptive scaling by the root second moment.

Q8. Step 1 Gradient: Adam and AdamW updates. Consider the scalar objective

$$f(\theta) = \theta^2, \quad \nabla_{\theta} f(\theta) = g_t = 2\theta_t.$$

Use the following hyperparameters unless otherwise specified:

$$\theta_0 = 10, \quad \alpha = 0.1, \quad \lambda = 0.01, \quad \beta_1 = 0.9, \quad \beta_2 = 0.99, \quad \epsilon = 10^{-8}, \quad m_0 = v_0 = 0.$$

Compute all quantities for $t = 1$.

- (a) Compute g_1, m_1, v_1 .
- (b) Compute bias-corrected $\hat{m}_1, \hat{v}_1$ (feel free to refer to Algorithm Block provided in Q1).
- (c) Compute $\theta_1^{\text{Adam}} = \theta_0 - \alpha \frac{\hat{m}_1}{\sqrt{\hat{v}_1 + \epsilon}}$ (step 1 update of Adam).
- (d) Compute $\theta_1^{\text{AdamW}} = \theta_1^{\text{Adam}} - \alpha \lambda \theta_0$ (step 1 update of AdamW).

Solution. Evaluate the first gradient at θ_0 (in general, $g_t = 2\theta_{t-1}$ for an update indexed by t).

- (a) $g_1 = 20, \quad m_1 = 0.1(20) = 2, \quad v_1 = 0.01(20^2) = 4.$
- (b) $\hat{m}_1 = 2/(1 - 0.9) = 20, \quad \hat{v}_1 = 4/(1 - 0.99) = 400.$
- (c) $\theta_1^{\text{Adam}} = 10 - 0.1 \frac{20}{20 + 10^{-8}} \approx 9.9.$
- (d) $\theta_1^{\text{AdamW}} = \theta_1^{\text{Adam}} - 0.1(0.01)(10) \approx 9.89.$

Q9. Step 2 Gradient: Adam continuation ($\beta_2 = 0.99$). Compute the second update step for Adam (you don't have to compute for AdamW). In other words, compute θ_2^{Adam} .

Solution. Continuing from Adam's $\theta_1 \approx 9.9$ (not AdamW's value),

$$\begin{aligned} g_2 &= 2\theta_1 \approx 19.8, \\ m_2 &= 0.9(2) + 0.1(19.8) = 3.78, \\ v_2 &= 0.99(4) + 0.01(19.8^2) = 7.8804, \\ \hat{m}_2 &= \frac{3.78}{1 - 0.9^2} \approx 19.894737, \quad \hat{v}_2 = \frac{7.8804}{1 - 0.99^2} = 396, \\ \theta_2^{\text{Adam}} &= 9.9 - 0.1 \frac{19.894737}{\sqrt{396} + 10^{-8}} \approx \boxed{9.800025}. \end{aligned}$$

Q10. Conceptual reflection. Keep the baseline at $\beta_2 = 0.99$. Now imagine we set $\beta_2 = 0.5$ (all else unchanged). Without redoing the full calculation, explain qualitatively what changes at $t = 1$ and $t \geq 2$, and how this would affect stability vs. responsiveness.

Solution. At $t = 1$, v_1 increases when β_2 decreases, but $\hat{v}_1 = (1 - \beta_2)g_1^2/(1 - \beta_2) = g_1^2$, so the corrected moment and parameter update are unchanged. For $t \geq 2$, $\beta_2 = 0.5$ gives recent squared gradients more weight; in this example the gradients decrease, so the second-step denominator is slightly smaller and the update is slightly larger. The estimate responds faster to changes but smooths noise less, so step sizes can fluctuate more than with $\beta_2 = 0.99$.